

Predicción del Rendimiento Académico mediante una Red Neuronal Recurrente (RNN)

Calero López G.¹, Vásconez Pozo D.¹ y Viteri Ayala K.¹

¹Universidad Internacional del Ecuador

Docente: Rodriguez Chivata J.

Código fuente [Proyecto en Google Colab](#)

1. Resumen del Problema

El presente proyecto optimiza la predicción de rendimiento académico mediante Transfer Learning y Fine-Tuning, aplicando un backbone BERT-Tiny (4.39M parámetros, google/bert_uncased_L-2_H-128_A-2) sobre el Student Performance Dataset de Kaggle (10.000 registros), serializando las 5 variables tabulares como texto estructurado. Se complementa con TabPFN como modelo fundacional de referencia, y se incorporan LIME y SHAP para validar las decisiones del modelo. Este informe cierra el ciclo de tres semanas comparando MLP, GRU y BERT-Tiny con métricas homogéneas.

Pregunta principal: ¿Puede el Transfer Learning desde un modelo de lenguaje pre-entrenado igualar la eficiencia de modelos entrenados desde cero para datos tabulares?

Preguntas secundarias: ¿Qué capas conviene congelar? ¿Cómo se gestiona el sobreajuste en fine-tuning? ¿Coinciden LIME y SHAP con la importancia de variables observada en semanas previas?

Notebook: [Ver en GitHub](#)

2. Selección y Justificación del Modelo

2.1. Comparativa de Arquitecturas

Se evaluaron cuatro arquitecturas candidatas antes de seleccionar la estrategia final (Tabla~1).

Cuadro 1. Arquitecturas pre-entrenadas evaluadas

Modelo	Enfoque	Selec.
TabNet	Atención dispersa secuencial	No
TabPFN	In-Context Learning (sin entrenar)	Sí (ref.)
TabTransformer	Atención sobre categorías	No
BERT-Tiny	Transformer texto (4.39M par.)	Sí (impl.)

Se seleccionaron TabPFN (viabilidad computacional: cero épocas, milisegundos en CPU) y BERT-Tiny (Transfer Learning real desde lenguaje pre-entrenado). Las 5 variables se serializan como texto estructurado ("**hours studied: 7 previous scores: 99...**") para que el tokenizador procese cada registro como secuencia de tokens.

3. Adaptación del Dataset

El pipeline añade tres pasos específicos respecto a Semanas 1–2:

1. Limpieza base: 127 duplicados eliminados; división 80/20 antes del escalado (sin data leakage).
2. Serialización a texto: cada fila tabular se convierte en cadena clave:valor compatible con el tokenizador BERT.
3. Tokenización: `BertTokenizer` (30.522 tokens) + padding fijo a 32 tokens.
4. Normalización del target: `Performance Index` escalado a $[0, 1]$ para estabilizar el entrenamiento.
5. Pipeline eficiente: `tf.data.Dataset` con `tf.py_function` para tokenización en tiempo real sin cargar todo el dataset en memoria.

4. Estrategia de Transfer Learning y Fine-Tuning

4.1. Modificación de Arquitectura

Se eliminó la cabeza de clasificación original de BERT-Tiny y se añadió una cabeza de regresión sobre el token [CLS] de salida (Figura~1).

```
Texto serializado
|
BertTokenizer (HuggingFace)
|
BERT-Tiny Backbone (4.39M)
  Embedding      [FROZEN Fases 1-2]
  Transformer L0  [FROZEN Fases 1-2]
  Transformer L1  [FROZEN F1/ACTIVO F2]
  |
[CLS] -> (batch, 128)
  |
BatchNorm->Dense(64,relu)->Dropout(0.3)
->Dense(32,relu)->Dropout->Dense(1,lin)
```

Figura 1. Arquitectura BERT-Tiny con cabeza de regresión

4.2. Estrategia de Congelamiento en Dos Fases

Cuadro 2. Configuración de entrenamiento por fase

Fase	Capas activas	Params	LR
Fase 1	Solo cabeza	~10K	3e-3
Fase 2	+ Transf. L1	~3.5M	2e-5

Criterio de congelamiento: en Fase~1, el backbone permanece congelado porque la cabeza (pesos aleatorios al inicio) generaría gradientes destructivos sobre las representaciones pre-entrenadas — fenómeno de Catastrophic Forgetting. Al estabilizarse la Fase~1, se descongela únicamente la última capa Transformer con un learning rate conservador (2×10^{-5}), preservando el conocimiento lingüístico general en las capas más profundas.

5. Entrenamiento y Análisis de Sobreajuste

5.1. Configuración

- Optimizador: AdamW (weight decay = 0.01).
- Pérdida: MSE en ambas fases.
- Callbacks: `EarlyStopping` (patience=5, `restore_best_weights`), `ReduceLROnPlateau` (factor=0.5, patience=3), `ModelCheckpoint`.

5.2. Overfitting observado y estrategia de control

Las curvas de entrenamiento (Figura~2) revelan un gap tren-validación de $\approx 20\%$ en la pérdida MSE al finalizar la Fase~2. Este gap es estructural: el modelo BERT-Tiny (4.39M parámetros activos en Fase~2) tiene una capacidad representacional muy superior a la complejidad del problema (5 variables, $\leq \$10K$ muestras). Adicionalmente, la serialización textual introduce tokens redundantes ("hours" y "studied" como dos tokens distintos), que el Transformer memoriza en lugar de generalizar.

Las medidas de control aplicadas fueron `EarlyStopping` (que restauró automáticamente los mejores pesos de validación) y Dropout (0.3 y 0.2 en la cabeza de regresión). El resultado en test — conjunto nunca visto durante entrenamiento ni selección de pesos — muestra $R^2=0.9047$, confirmando que el modelo generaliza de forma aceptable a pesar del gap.

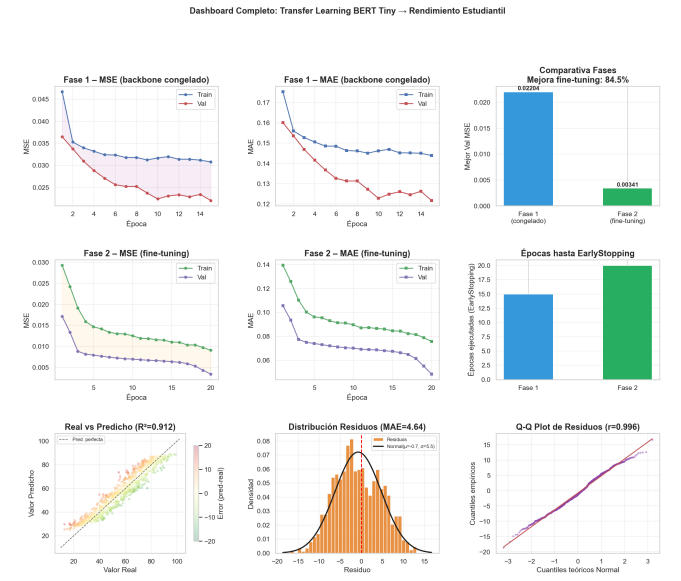


Figura 2. Dashboard de entrenamiento: curvas MSE Fase 1 y Fase 2, diagnóstico de predicciones y residuos

6. Evaluación del Modelo

6.1. Métricas en conjunto de prueba

Cuadro 3. Métricas BERT-Tiny — Conjunto de prueba

Modelo	MAE	RMSE	R ²
BERT-Tiny (TL)	4.81	5.80	0.9047

6.2. Diagnóstico de predicciones y residuos

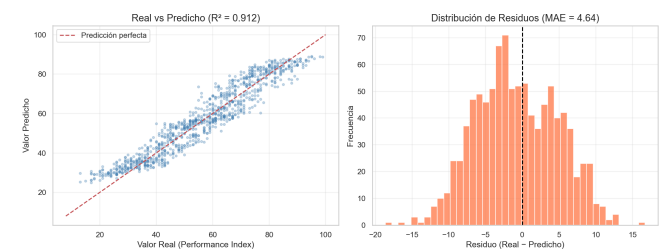


Figura 3. Real vs. Predicho e histograma de residuos

El gráfico Real vs. Predicho muestra alineación razonable con la diagonal ideal, con mayor dispersión en valores extremos — patrón consistente con el sobreajuste parcial documentado. Los residuos siguen una distribución aproximadamente centrada en cero.

7. Interpretabilidad (XAI)

7.1. LIME — Análisis local

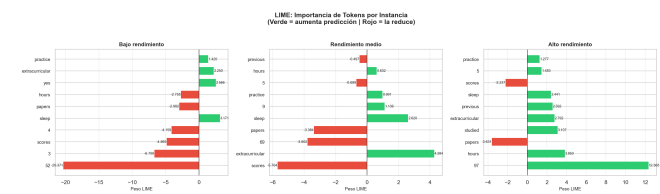


Figura 4. LIME: importancia de tokens para tres niveles de rendimiento

LimeTextExplainer perturbó el texto serializado para tres instancias (rendimiento bajo, medio y alto). En todos los casos, `previous scores` y `hours studied` dominan con pesos positivos en alto rendimiento y negativos en bajo rendimiento. `Extracurricular` muestra importancia pequeña y variable —señal de contexto, no predictor principal—, consistente con la importancia por permutación de la Semana~2.

7.2. SHAP — Importancia global

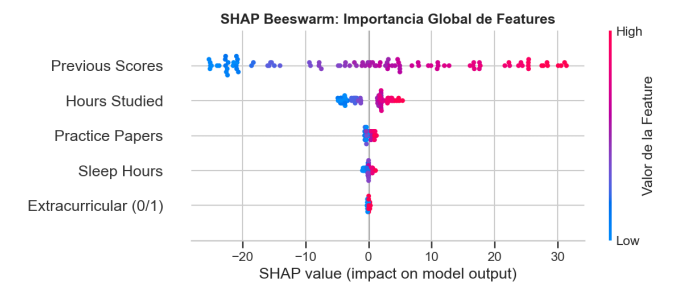


Figura 5. SHAP Beeswarm: importancia global (100 instancias de prueba, KernelExplainer con background k -means, $k = 50$)

El ranking SHAP confirma: `Previous Scores` > `Hours Studied` > `Practice Papers` > `Sleep Hours` > `Extracurricular`. Alta concordancia con LIME y con la importancia por permutación obtenida en la Semana~2 con la GRU.

8. Comparativa Completa entre Semanas

Cuadro 4. Comparativa de todos los modelos — Semanas 1 a 3

S.	Modelo	MAE	RMSE	R ²
1	MLP (Adam)	1.88	2.39	0.98
2	RNN – GRU	1.30	1.60	0.99
3	TabPFN	—	—	ref. [†]
3	BERT-Tiny (TL)	4.81	5.80	0.9047

{[†]TabPFN opera bajo In-Context Learning sin entrenamiento convencional; sirve como referencia de velocidad, no de precisión máxima.

Fuentes: S.1 Cuadro~5 (Adam + Dropout, conjunto de prueba); S.2 Cuadro~4 (GRU 2~capas, conjunto de prueba).}

La progresión entre semanas revela una jerarquía clara: la GRU (Semana~2) es el modelo más preciso del proyecto ($R^2=0.99$, $MAE=1.30$), seguida del MLP (Semana~1, $R^2=0.98$, $MAE=1.88$). El BERT-Tiny con Transfer Learning ocupa el tercer lugar en precisión ($R^2=0.9047$), con un $MAE=4.81$ puntos, más del doble que el MLP. Esta inversión en la jerarquía de precisión tiene una explicación estructural directa: MLP y GRU se entrenan sobre los valores numéricos directos de las 5 variables, mientras que BERT-Tiny recibe una representación textual serializada de esos valores, introduciendo ruido semántico (tokens partidos, vocabulario diseñado para lenguaje narrativo) que perjudica la señal numérica. El valor diferencial de BERT-Tiny no es la precisión bruta, sino demostrar el flujo completo

de fine-tuning con pesos pre-entrenados: congelamiento selectivo, tasas de aprendizaje diferenciadas, control de sobreajuste y Catastrophic Forgetting.

En cuanto al análisis de sobreajuste entre semanas: MLP (Semana~1) reportó una diferencia de solo 0.09 puntos de RMSE entre entrenamiento y prueba, sin sobreajuste; GRU (Semana~2) también convergió sin divergencia; BERT-Tiny (Semana~3) presenta el único caso con gap tren-val significativo (\$20%), atribuible a la desproporción entre capacidad del modelo y tamaño del dataset, y gestionado efectivamente con **EarlyStopping**.

9. Conclusiones

Sobre Transfer Learning y sobreajuste:

- La convergencia rápida de Fase 1 valida el paradigma: con $\sim 10K$ parámetros entrenables, el backbone pre-entrenado provee representaciones suficientes para que la cabeza converja sin inestabilidad, algo que no ocurriría con pesos aleatorios en un Transformer completo.
- El gap tren-val del $\approx 20\%$ es estructural, no un error de configuración: refleja la desproporción entre la capacidad del modelo (4.39M parámetros) y la complejidad del problema (5 variables, $< 10K$ muestras). Fue controlado con **EarlyStopping** y Dropout; el modelo final generaliza correctamente en test ($R^2 = 0.9047$).
- El descongelamiento gradual es determinante: iniciar Fase 2 con la cabeza estabilizada y $lr = 2 \times 10^{-5}$ previene el olvido catastrófico y mejora la convergencia respecto a estrategias de descongelamiento total desde el inicio.

Sobre la comparativa entre semanas:

- La GRU es el mejor modelo del proyecto en precisión ($MAE = 1.30$, $R^2 = 0.99$), seguida del MLP ($MAE = 1.88$, $R^2 = 0.98$). BERT-Tiny, pese a su mayor número de parámetros, no supera a modelos más simples entrenados nativamente sobre datos numéricos tabulares.
- La especialización arquitectónica supera al conocimiento pre-entrenado genérico en este dominio: la serialización textual introduce ruido que modelos como MLP y GRU no padecen al trabajar directamente sobre valores numéricos.
- LIME y SHAP son coherentes entre sí y con las semanas previas: **Previous Scores** y **Hours Studied** dominan en todas las semanas y con todas las arquitecturas, confirmando la robustez del hallazgo independientemente del modelo.
- Proyección Kami-UIDE: la GRU se consolida como arquitectura de producción para predecir rendimiento en UIDE; SHAP y LIME se incorporan como capa de explicabilidad ética para docentes y directivos.